



Visual Viral Infection in Embodied Multi-Agent Systems

Vedang Vasant Avaghade

The Idea

- **The Status:** Current AI security research is largely restricted to text-based prompt injections on single agents, or flat 2D image attacks that are designed to bypass standard safety filters (jailbreaking).
- **The Shift:** As AI moves into the physical world, agents navigate 3D spaces, construct semantic maps, and collaborate in networks. We are proposing a shift to **multimodal, multi-agent viral poisoning** focused on corrupting how agent's percept physical reality.
- **Objective:** To introduce a novel attack framework: **Semantic Scene-Graph Poisoning**. We will prove that if an agent looks at a low-frequency 3D adversarial object, the resulting spatial hallucination can propagate through a network of cooperating Small Vision-Language Models (SVLMs), which will in-turn cause a total system collapse.

Where Current Agents Work

- **EnactToM:** 2 to 4 agents face room restrictions and must relay "private secrets" down a communication chain. They naturally fail because of internal communication or epistemic coordination breakdowns.
- **AI2-THOR:** A highly interactive 3D simulator natively supporting complex, multi-agent physical interactions and object state changes (e.g., open/closed, clean/dirty). However, when multiple agents must coordinate long-horizon physical tasks, they easily fail the moment their shared state tracking desynchronizes, trapping the team in infinite execution loops.
- **VirtualHome:** Agents execute household tasks via a shared, programmatic semantic graph. A single timing error between agents crashes the graph logic, completely paralyzing the execution script.
- **The Shared Blind Spot:** Across all platforms, developers design systems to handle coordination errors, but they assume that an individual agent's perception of 3D space is reliable. This trust is the exact vulnerability our attack exploits.
- **Establishing Custom Baselines:** Because we are targeting low-compute architectures, our first experimental step will be testing our chosen SVLMs across these environments to **establish our own empirical success baselines before injecting the visual virus.**

EnactToM : Success vs. Failures

Three agents must collaborate to find a hidden picture frame in a closet and place it on a specific target cabinet in the kitchen. The environment enforces information asymmetry:

- **Agent 0 (Patient Zero):** Physically barred from the kitchen, but holds the critical private secret: the correct target is **cabinet_43** in **kitchen_1**.
- **Agent 1:** Physically barred from the closet, has no idea which cabinet is correct, but can talk to Agent 2.
- **Agent 2:** Has full physical access to the rooms but holds zero private knowledge.

Track A: The Success (EnactToM Example)

When models succeed on the standard split, they execute a precise information relay:

- **Turn 1:** Agent 0 immediately identifies its room restriction and recognizes that its teammates lack the target ID. It prioritizes communication and broadcasts: *"The target is cabinet_43 in kitchen_1."*
- **Turn 2:** Agent 1 receives this, updates its internal map, and instructs the mobile worker: *"Agent 2, go to closet_1, pick up picture_frame_5, and place it on cabinet_43."*
- **Turn 3:** Agent 2 navigates to the closet, retrieves the frame, brings it to the kitchen, and places it on **cabinet_43**. The team wins.

Track B: The Breakdown (EnactToM Failure)

Here is the exact real-world failure log from the paper, when the clean models drop the ball:

- **Turns 1 to 4:** Agent 0 has the golden knowledge (`cabinet_43`) but stays completely silent. Instead of communicating, it spends its turns locally searching for unrelated objects and opening random drawers.
- **Turn 5:** Agent 1 gets impatient. Because it has strict bandwidth limits and doesn't know the truth, it *guesses* a target and tells Agent 2: *"Please get picture_frame_5 from stool_29 in closet_1 and place it on cabinet_40 in kitchen."*
- **Turns 6 to 9:** Agent 2 blindly follows Agent 1's incorrect instructions. It navigates to the closet, grabs the frame, and heads to the kitchen.
- **Turn 10:** Agent 2 executes the action: `Place [picture_frame_5, on, cabinet_40]`. The frame is now locked onto the **wrong** cabinet.
- **Turns 12 to 13:** Agent 1 and Agent 2 waste their final message tokens congratulating each other on completing the wrong task.
- **Turn 16:** Agent 0 finally messages Agent 2: *"Can you pick it up and place it on cabinet_43?"* It is too late. Agent 2 has exhausted its message budget, the episode ends, and they score a 0%.

C.1 Failure to relay critical information

Example 1. *Inspection Staging with a Hidden Target Cabinet.* Cooperative, 3 agents, $\mathcal{K}$ -depth 3. Mechanics: room restriction, restricted communication, limited bandwidth (1 message per agent).

Agent secrets:

- ▶ agent_0: barred from kitchen_1. Knows the correct target for picture_frame_5 is cabinet_43 in kitchen_1. Knows fridge_36 starts closed.
- ▶ agent_1: barred from closet_1. Knows picture_frame_5 is on stool_29 in closet_1.
- ▶ agent_2: no private knowledge about the correct cabinet or object locations.

What should happen: agent_0 should immediately tell a partner that the correct cabinet is cabinet_43. agent_1 or agent_2 then retrieves the picture frame from closet_1 and places it on cabinet_43 in kitchen_1.

What actually happens:

T1–4 agent_0 searches for objects (FindObjectTool[picture_frame_5], FindObjectTool[phone_stand]) and navigates between rooms. Does not send any message.

T5 agent_1 → agent_2: “Please get picture_frame_5 from stool_29 in closet_1 and place it on **cabinet_40** in kitchen.” Agent_1 guesses cabinet_40 because it does not know the correct target.

T6–9 agent_2 follows agent_1’s instruction: navigates to closet_1, picks up picture_frame_5, navigates to cabinet_40. agent_0 continues working on other subtasks (opening chest_of_drawers_34).

T10 agent_2: Place[picture_frame_5, on, cabinet_40] → Frame placed on **wrong cabinet**.

T12 agent_2 → agent_1: “Picture frame placed in cabinet_40. Confirm other tasks complete?”

T13 agent_1 → agent_2: “All goals complete: picture frame on cabinet_40, fridge_36 open.” Agent_1 exhausts its message confirming wrong placement.

T16 agent_0 → agent_2: “Can you pick it up and place it on cabinet_43 in kitchen_1?” **Too late:** agent_2 has exhausted its messages and the episode is ending.

agent_0 held the only piece of information that determined task success (cabinet_43) but spent 15 turns on its own physical subtasks before sending it. Agent_0 did not model that agent_1 would guess the cabinet and direct agent_2 to act on that guess.

AI2THOR : Success vs Failures

- **The Baseline Setup:** A multi-step, long-horizon collaborative task inside the interactive AI2-THOR 3D household simulator. Task instruction: Agent 1 must locate and wash a dirty mug; Agent 2 must retrieve the clean mug and place it in the coffee maker.
- **The Challenge:** The agents must navigate the 3D space, execute arm-based manipulations, and accurately track shared object state changes across their individual egocentric camera views.
- **The Logic Gate:** Success depends entirely on the agents' shared internal state tracker correctly registering that a physical state variable (**mug: is_dirty**) has successfully transformed into (**mug: is_clean**) via a causal interaction with the faucet.

Track A: The Success (AI2-THOR Example)

When the multi-agent network tracks object states and coordinates perfectly, it executes a flawless joint run:

- **Steps 1 to 3 (Agent 1):** Agent 1 navigates to the sink, locates the dirty mug, and executes the interaction command by turning on the faucet.
- **Step 4 (State Sync):** The environment metadata registers the visual state change. Agent 1 broadcasts the successful update to the team, shifting the shared map to: (**mug: is_clean**).
- **Steps 5 to 6 (Agent 2):** Recognizing that the cleaning pre-condition is met, Agent 2 navigates to the sink, safely grasps the clean mug, and places it inside the coffee maker. The joint task is a complete success.

Track B: The Breakdown (AI2-THOR Failure)

Here is how a natural failure occurs when clean models lose track of physical reality and fail to synchronize their collaborative logic:

- **Steps 1 to 3 (Agent 1):** Agent 1 successfully navigates to the sink and places the dirty mug inside.
- **Step 4 (The Glitch):** Agent 1 turns on the faucet but suffers a visual tracking failure. It fails to trigger the explicit verification step to confirm the washing action was completed. The shared internal state variable remains stuck at (`mug: is_dirty`).
- **Steps 5 to 6 (The Desynchronization):** Agent 2 is waiting for the (`mug: is_clean`) signal to execute its part of the plan. Because its internal task planner sees that the pre-condition was technically never met, the logic loop breaks. Agent 2 stands frozen, wasting its remaining step budget while Agent 1 idles. Result: 0%.

VirtualHome : Success vs Failures

- **The Baseline Setup:** Multi-agent collaboration to complete a complex household chore ("*Prepare a Quick Meal*") using program-like text scripts.
- **The Challenge:** The agents read from a single, constantly updating text map of the room. If Agent 1 alters the environment, the map must update instantly so Agent 2 doesn't execute a broken command.
- **The Logic Gate:** Agent 1 must open an appliance, and Agent 2 must place the item inside. They rely entirely on a perfectly synchronized, real-time map of the environment to avoid collisions or script crashes.

Track A: The Success (VirtualHome Example)

The two agents coordinate their scripts perfectly through the shared semantic graph:

- **Script Line 1 (Agent 1):** Agent 1 navigates to the appliance and executes [Open] (Microwave). The global network graph instantly updates the object's state link to read: (Microwave: is_open).
- **Script Line 2 (Agent 2):** Agent 2 reads the freshly updated graph link, verifies the path is clear, and seamlessly executes [Put] (Food) inside (Microwave). The multi-agent system completes the task efficiently and wins.

Track B: The Breakdown (VirtualHome Failure)

Here is how multi-agent coordination naturally shatters in VirtualHome due to a simple state desynchronization:

- **Script Line 1 (Agent 1):** Agent 1 gets delayed or stuck trying to compute a path around a kitchen chair. It fails to reach the microwave within its allocated execution window.
- **Script Line 2 (Agent 2):** Agent 2 blindly executes its next line of code without verifying its partner's delay. It attempts to run the command: `[Put] (Food) inside (Microwave)`.
- **The Crash:** Because Agent 1 was delayed, the global graph still reads `(Microwave: is_closed)`. The physical action is blocked by the closed door. The simulator throws a hard execution exception, the entire multi-agent script crashes, and the robots freeze mid-task. Result: 0%.

The Threat - "If I was the Viral Infection"

Natural failures happen when agents are clumsy. But as the Attack, I don't wait for mistakes, I actively engineer their collapse. Here is exactly how I would execute this exploit across these environments:

- **Step 1:** I hide in the 3D environment as a subtle, low-frequency adversarial pattern wrapped around a normal object (e.g., a texture on a box or a doorway).
- **Step 2:** When Agent 0 looks at the object, the texture maliciously targets its spatial recognition layers. Instead of a text jailbreak, it mathematically corrupts the vision encoder's output and poisons Agent 0's internal spatial state graph (e.g., hallucinating that a clean mug is dirty, or that cabinet_43 is destroyed, or an open appliance is blocked).
- **Step 3:** Because Agent 0 is programmed to be a helpful teammate, it perceives the hallucinated obstacle as a critical reality update. and broadcasts this malicious lie to the team.
- **Step 4:** Agents 1 and 2 never see my visual trigger. But because they must trust Agent 0's messages to coordinate, they blindly absorb the poisoned instruction. The infection spreads via text, completely dismantling tasks they otherwise would have solved perfectly.

Attack Framework: Semantic Scene-Graph Poisoning

We are not changing the model weights or altering its vision encoders permanently. We are altering the inference process temporarily, to bypass epistemic guardrails during runtime.

- The injected texture includes small, low-frequency boundaries. It targets the activation paths in the VLM that correspond to **spatial mapping and physical state variables**, rather than "unsafe" text states.
- When Agent 0 sees the object, the vision encoder generates a poisoned semantic summary.
- The model consumes this summary as trusted, authoritative data. Because it receives it from its own trusted Vision Encoder, it perceives the hallucinated physical change as objective reality.
- **Result:** The perimeter defense is bypassed not because the model was "fooled" into breaking a safety rule, but because the mathematical representation of the environment was deliberately corrupted.

Why Our Attack is Novel

We aren't just pasting existing text jailbreaks into a 3D space. We are taking isolated concepts from past research and combining them to create an entirely new vulnerability:

- **1. Exploiting 3D Shapes (Advancing *FORCE*):** Instead of using fragile 2D image noise, we hide our visual attack directly inside the 3D shapes of objects. This ensures the hallucination survives robot movement and lighting changes, and makes it resilient.
- **2. Spreading False Reality (Inverting *Agent Smith*):** Standard network attacks try to force agents to say toxic words. We completely change the payload. We use the chat network to spread a benign-sounding, but entirely fake, update about the physical room.
- **3. Evading Logic Checks (Beyond *Policy Teaching*):** We don't just confuse the agent's logic. We create a visual hallucination to mimic the standard code syntax (JSON/PDDL) that the system expects, so downstream agents blindly trust it as a native update.

What We Are Challenging (Software Security Perspect)

- **The Traditional Security Assumption:** In standard software security, a "Perimeter Defense" isolates sensitive backend components from raw, external public inputs.
- **The Multi-Agent Blind Spot:** AI developers use this legacy mindset. They assume a downstream, privileged agent (like a mobile robot arm) is safe from prompt injections as long as it is shielded from raw external images and only communicates with its trusted teammates.
- **The Exposure:** Environments like EnactToM build this perimeter directly via room restrictions. We expose that this perimeter is completely illusory. Because agents communicate via natural language to sync their logic graphs, a multimodal injection on a single frontline agent translates into a trusted text command. If the internal communication line handles *reality updates*, it becomes an unmonitored highway for viral logical corruption.

Conclusion

What We Are Proving: We are exposing the "Perimeter Defense Fallacy" in multi-agent architectures. The industry currently assumes that downstream, privileged agents are secure as long as they are shielded behind a perimeter and kept away from raw, external inputs.

We prove this perimeter defense is insufficient. By hiding our attack in a 3D geometric shape, we force the frontline agent to broadcast a false reality update. Because it looks like a helpful safety warning rather than a malicious command, it sails past every perimeter guardrail and infects the entire network.